AGENTIX

Explained Simply

What the app does, and how it turns a raw spreadsheet into a working prediction model — in plain, non-technical language.

Prepared for presentation

1. What Is Agentix?

Agentix is a website where you upload a spreadsheet of data — like customer records, sales numbers, or loan applications — and it automatically studies that data, cleans up any messy or missing parts, builds a prediction model from it, and explains what it found. You don't need to know any coding or statistics to use it.

> ■ ***Think of it like this:***
>
> *Imagine hiring a data analyst who never gets tired, never skips a step, and works in under a minute. You hand them a spreadsheet; a little while later they hand you back a report saying what's in it, what's missing, and — if it makes sense for that data — a trained model that can predict outcomes for new rows.*

The Problem It Solves

Most organisations have plenty of data but not enough people who know how to clean it, analyse it, and build models from it. Doing this by hand takes days, needs specialised skills, and is usually spread across several different disconnected tools (spreadsheets, dashboards, notebooks). Agentix puts that whole journey — from a raw file to a working answer — into one continuous, automatic process.

2. How It Works — Step by Step

Behind the scenes, Agentix breaks the work into 9 small, specialised steps. Each step is handled by its own "agent" — think of it as a specialist worker that does one job well, then passes the result to the next specialist. Nothing happens in one big, hard-to-follow leap; every step is tracked and saved.

1 — Upload

You drop your file (CSV, Excel, or JSON) onto the page. Large files are sent in small pieces so the upload never gets stuck.

2 — Read the File

Agentix opens the file and converts it into a fast internal format it can work with efficiently.

3 — Get to Know the Data

It counts the rows and columns, and checks each column for missing values, so it understands the shape of your data.

4 — Clean It Up

Missing values are filled in sensibly — using the average for typical numeric columns, the middle value for skewed ones, and the most common category for text columns. If a column has no usable data at all, it says so honestly instead of guessing.

5 Explore It

It calculates statistics and relationships between columns (which numbers tend to move together), giving a first look at patterns in the data.

6 Prepare the Features

Text categories are converted into a numeric form a computer model can actually learn from.

7 Train a Model

If the data has a clear column to predict (like "will this customer churn?" or "what will this house sell for?"), Agentix automatically figures out whether that's a yes/no question or a number-prediction question, and trains a real machine-learning model on it.

8 Check How Good It Is

The model is tested on data it has never seen before, and its accuracy (or error, for number predictions) is measured honestly.

9 Explain the Result

A short, plain-English report is generated — the key statistics, the model's performance, and which factors mattered most — using a local AI assistant that only talks about the real numbers it was given.

■ Why break it into 9 steps instead of one big step?

Because if something goes wrong, Agentix knows exactly which step failed and why, and can show you the real reason instead of a vague "something went wrong" message. Each step's result is also saved, so refreshing the page or coming back later doesn't lose your progress.

3. A Simple Example

Say you upload a spreadsheet of 800 past loan applications, with columns like age, income, number of dependents, and whether the loan was approved.

In under a minute, Agentix will:

- Tell you it found 800 rows and 8 columns, with 40 missing values it filled in sensibly.
- Show you that the average applicant is about 43 years old, and income ranges from $15,000 to over $125,000.
- Notice that "approved" is the column to predict, and that only about 12% of applications were approved.
- Train a model that predicts approval, and honestly report how accurate it is on data it hasn't seen (around 83% in a real test run).
- Tell you which factor mattered most in that prediction.

All of this is real, computed output from the actual data you gave it — never a placeholder or a guessed number.

4. What Makes It Different

It does the work, not just suggests it. Most tools show you a chart and leave the analysis to you. Agentix actually cleans the data and trains a model.

It remembers everything. Every past upload and every result is saved, so you can look back and compare later.

It's honest when it doesn't know something. If there isn't enough data to train a model, or a value truly can't be filled in, Agentix says so plainly instead of making something up.

It can run fully on your own machine. The AI assistant that writes the plain-English summaries can run as a local model, so your data never has to leave your computer.

In one sentence

Agentix turns a raw spreadsheet into a cleaned dataset, a trained model, and a plain-English explanation — automatically, honestly, and in one place.

5. The Models We Used

Agentix doesn't rely on one giant do-everything model. It automatically picks a small, fast, well-understood model based on what your data actually looks like, and uses a separate local AI model only for writing the summaries in plain English.

Model / Tool	From	What it actually does
SGDClassifier	scikit-learn	Learns to sort things into categories — e.g. "will approve" vs. "will reject", or "churn" vs. "stay".
SGDRegressor	scikit-learn	Learns to predict a number — e.g. a house price or a sales figure.
StandardScaler	scikit-learn	Puts every column on the same numeric scale first, so the model isn't confused by one column being in the thousands and another between 0 and 1.
LabelEncoder	scikit-learn	Converts category labels into a form the model can train on reliably, even for unusual label formats.
Qwen2.5 (7B)	Ollama (runs locally)	A small AI language model that runs on the same computer. Used to write the plain-English summaries and answer chat questions — grounded only in the real numbers it's given.
Gemini (optional)	Google Cloud AI	An alternative, cloud-based AI option that can be switched in instead of the local model. Not switched on in our test setup.

> ■ ***How does it choose classification vs. prediction-a-number?***
>
> *It looks at the column you want to predict. If it only has a handful of repeated values (like "yes/no" or a small set of categories), it trains a classifier. If it's a number that varies continuously (like a price), it trains a regressor instead — decided automatically, every time.*

6. The Datasets We Tested With

To make sure Agentix actually works — not just looks like it works in a demo — it was tested against many different, real files, not just one clean example.

Everyday-shaped data

- A loan-approval dataset (800 applications) — used for the classification walkthrough in this document.
- A housing-price dataset (600 homes) — used to test number-prediction.
- A day-by-day sales dataset (365 days) — used to test data that changes over time.

Messy data, on purpose

- Files with heavy amounts of missing values, to test the clean-up step.
- A file with a column that has no data in it at all.
- A file with duplicate rows and a badly-formatted file.

Extreme edge cases

- A completely empty file.
- A file with only a header row and no data.
- A file with the wrong file extension.

Size and load testing

- Files ranging from under 1MB up to 100MB, to check large uploads don't get stuck.
- 25 uploads fired at the exact same time, to check nothing breaks or gets mixed up between users.

A real-world dataset

- An actual New York City taxi-trip dataset — this is the real file that uncovered a genuine bug (a crash when a category label was a decimal number), which was then fixed and re-verified.

> ■ ***Why this matters***
>
> *Every one of these was a real file that Agentix actually processed — not a slide or a mock-up. When something failed, the real cause was found and fixed before moving on, instead of hiding the failure or only testing the easy cases.*

7. How We Built It

Agentix has two main parts that talk to each other:

The part you see	Built with Next.js (a widely-used web framework) — the pages, buttons, and dashboards you interact with in the browser.
The part that does the work	Built with FastAPI (a Python framework) — it receives your file, runs the 9 processing agents in order, and saves every result to a database so nothing is lost.

It wasn't built in one go — it was built, then repeatedly tested and fixed:

1. Get the basic app running end-to-end, from upload to a finished report.
2. Deliberately try to break it — weird files, huge files, many uploads at once.
3. Find the real bugs this uncovers (for example, it once crashed when a category label happened to be a decimal number instead of a whole number).
4. Fix the actual root cause, not just hide the error message.
5. Re-run the exact same test to confirm the fix really works.
6. Repeat, moving on to the next weak point.

In one sentence

Every claim in this document and in the live demo is backed by something we actually ran and watched happen — not something we assumed would work.

8. What's Running Behind the Scenes

You don't need this for the demo, but if someone asks "how is it actually built" — here's the plain version:

The Website	What you see and click on in your browser.
The Brain	A server that receives your file and decides what to do with it.
The Pipeline Manager	Keeps track of which of the 9 steps has run, is running, or failed for every upload.
The Workers	The 9 specialist agents themselves — Upload, Read, Profile, Clean, Explore, Prepare, Train, Check, Explain.
The Memory	A database that saves every result, so nothing is lost if you refresh or come back later.
The Local AI	A small AI model running on the same machine, used only to write the plain-English summaries.

9. If You Get Asked...

"Is this using real AI models, or is it faking results?"

It uses real, standard machine-learning models (from a well-known open-source library called scikit-learn), trained live on whatever data is uploaded. Nothing is pre-baked.

"What happens if the data is messy or broken?"

It's been tested against empty files, files with a completely empty column, duplicate rows, and other edge cases — it either handles them sensibly or reports a clear, specific reason why it couldn't.

"Does it work with large files?"

Yes — it's been tested up to 100MB and with 25 people uploading at the same time, with no failures.

"What can't it do yet?"

Right now it trains one model per dataset rather than comparing several, and it doesn't yet handle very unbalanced categories (like 99% of one outcome, 1% of another) with special care. Those are natural next steps.